Contents

- **Arrays:** Defining an array - Processing an array - Multi dimensional arrays
- **Pointer Variables:** Definitions and initializations - Pointer operators - Pointer expressions and arithmetic
- Pointers and one dimensional arrays
- String operations.

Arrays

Dr. Aguru Aswani Devi

Array

- An array is a **collection of elements** of the **same data type** stored in **contiguous memory locations** and accessed using a common **name** and an **index**.

```
int m1 = 75, m2 = 82, m3 = 68, m4 = 91, m5 = 77;
```

```
int marks[5] = {75, 82, 68, 91, 77};
```

75 Index 0	**82** Index 1	**68** Index 2	**91** Index 3	**77** Index 4
marks[0]	**marks[1]**	**marks[2]**	**marks[3]**	**marks[4]**

Defining an array

- General syntax: `data_type array_name[size];`
- Examples:

```
int age[10];
float salary[5];
char grade[4];
```

- ✓ **data_type:** type of values stored.
- ✓ **array_name:** name used to access the array.
- ✓ **size:** maximum number of elements declared for the array.

Initializing an array

- **Assigning initial values to an array** can be done in 3 ways.

1. Declare and initialize together. `int a[5] = {10, 20, 30, 40, 50};`
2. Allow the compiler to determine the size. `int a[] = {10, 20, 30};`
3. Initialize selected elements. `int a[5] = {10, 20};`

- The remaining elements are initialized to zero, so the array contains: {10, 20, 0, 0, 0}. This zero-initialization rule applies to the unspecified elements of an array initialized with an initializer list.

- **What about int a[5];**

Accessing array elements

- Each element is accessed using the array name and an index.

```
int a[3] = {10, 20, 30};

printf("%d", a[0]);  // 10
printf("%d", a[1]);  // 20
printf("%d", a[2]);  // 30
```

- index = position − 1

Processing an array

- Processing means performing operations on the array elements, such as reading, displaying, calculating the sum, finding the largest element, or searching for a value.

```c
#include <stdio.h>
int main()
{
    int a[5], i;
    printf("Enter 5 numbers: ");
    for (i = 0; i < 5; i++)
        scanf("%d", &a[i]);
    printf("Array elements are: ");
    for (i = 0; i < 5; i++)
        printf("%d ", a[i]);
    return 0;
}
```

Practice Example - 1

```c
#include <stdio.h>
int main()
{
    int a[5], i;
    printf("Array elements are: ");
    for (i = 0; i < 5; i++)
        printf("%d ", a[i]);
    return 0;
}
```

Array elements are: -360986128 32765 -662315287 32684 -360986128

A garbage value occurs when a variable is declared but not initialized before use. The memory allocated to the variable may contain leftover data from previous programs or random values already present in that memory location, which the program interprets as the variable's value.

Practice Example - 2

```c
#include <stdio.h>
int main()
{
    int a[5] = {40, 50}, i;
    printf("Array elements are: ");
    for (i = 0; i < 5; i++)
        printf("%d ", a[i]);
    return 0;
}
```

Array elements are: 40 50 0 0 0

Practice Example - 3

```c
#include <stdio.h>
int main()
{
    int a[], i;
    printf("Array elements are: ");
    for (i = 0; i < 5; i++)
        printf("%d ", a[i]);
    return 0;
}
```

```
Compilation failed due to following error(s).
main.c: In function ‘main’:
main.c:4:9: error: array size missing in ‘a’
   4 |     int a[], i;
     |         ^
```

Practice Example - 4

```c
#include <stdio.h>
int main()
{
    int a[] = {}, i;
    printf("Array elements are: ");
    for (i = 0; i < 5; i++)
        printf("%d ", a[i]);
    return 0;
}
```

Array elements are: 0 981592224 32767 -2060073768 29684

Practice Example - 5

```c
#include <stdio.h>
int main()
{
    int a[] = {20, 30}, i;
    printf("Array elements are: ");
    for (i = 0; i < 5; i++)
        printf("%d ", a[i]);
    return 0;
}
```

Array elements are: 20 30 2 -2017306048 32765

Practice Example - 6

```c
#include <stdio.h>
int main()
{
    int a[] = {}, i;
    printf("Array elements are: ");
    for (i = 0; i < 5; i++)
        printf("%d ", a[i]);
    return 0;
}
```

Array elements are: 0 -2100831584 32764 437232856 32100

Practice Example - 7

```c
#include <stdio.h>
int main()
{
    int a[5] = {}, i;
    printf("Array elements are: ");
    for (i = 0; i < 5; i++)
        printf("%d ", a[i]);
    return 0;
}
```

Array elements are: 0 0 0 0 0

Sum and average of array elements

```c
#include <stdio.h>
int main()
{
    int a[5] = {10, 20, 30, 40, 50}, i, sum = 0;
    float average;
    for (i = 0; i < 5; i++)
        sum = sum + a[i];
    average = sum / 5.0;
    printf("Sum = %d\n", sum);
    printf("Average = %.2f\n", average);
    return 0;
}
```

```
Sum = 150
Average = 30.00
```

Dry Run

- Initially, sum = 0.
- The average is calculated as total sum divided by the number of elements.

Iteration	a[i]	New sum
1	10	10
2	20	30
3	30	60
4	40	100
5	50	150

Multidimensional array: Declaration and initialization

- An array whose elements are organized using multiple indices.
- The most commonly used multidimensional array is a two-dimensional (2D) array, which represents data in rows and columns, similar to a table or matrix.

```
data_type array_name[rows][columns];
```

- **Example:** marks of 3 students in 2 subjects.
- The first index identifies the row
- The second index identifies the column.
- marks[3][2] — 3 rows × 2 columns

```
int marks[3][2] = {
    {80, 90},
    {75, 85},
    {88, 92}
};
```

Multidimensional arrays

```
int marks[3][2] = {
    {80, 90},
    {75, 85},
    {88, 92}
};
```

Student	Subject 1 Index: 0	Subject 2 Index: 1
Index: 0	80	90
Index: 1	75	85
Index: 2	88	92

```
marks[0][0]   // 80
marks[0][1]   // 90
marks[1][0]   // 75
marks[1][1]   // 85
marks[2][0]   // 88
marks[2][1]   // 92
```

Processing a 2D array using nested loops

- The outer loop controls the rows, the inner loop controls the columns.

```c
#include <stdio.h>
int main()
{//Display a matrix
    int a[2][3] = {{1, 2, 3}, {4, 5, 6}}, i, j;;
    for (i = 0; i < 2; i++)
    {
        for (j = 0; j < 3; j++)
        {
            printf("%d ", a[i][j]);
        }
        printf("\n");
    }
    return 0;
}
```

```
1 2 3
4 5 6
```

Addition of two matrices

➢Two matrices can be added directly when they have the same dimensions.

```c
#include <stdio.h>
int main()
{
    int a[2][2] = {{1, 2}, {3, 4}}, b[2][2] = {{5, 6}, {7, 8}}, c[2][2], i,
j;
    for (i = 0; i < 2; i++)
    {
        for (j = 0; j < 2; j++)
            c[i][j] = a[i][j] + b[i][j];
    }
    for (i = 0; i < 2; i++)
    {
        for (j = 0; j < 2; j++)
            printf("%d ", c[i][j]);
        printf("\n");
    }
    return 0;
}
```

```
6 8
10 12
```

Pointers

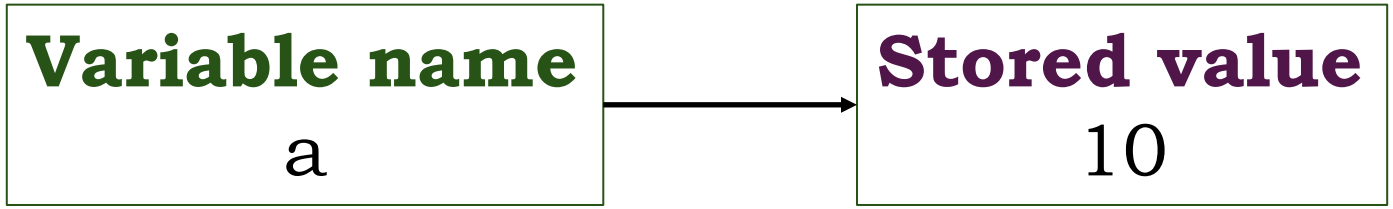
Dr. Aguru Aswani Devi

Variable

- A variable is a named memory location used to store a value that may change during program execution.

```
int a = 10;
```

- Here, a stores the value 10. The variable also has a memory address.



- a → value stored in the variable.
- &a → address of the variable.
- The address identifies where the variable is stored in memory.

Pointer

- A pointer is a variable that stores the memory address of another object, such as an integer or a character variable.
- a is an integer variable containing 10.
- p is a pointer to an integer.

```
data_type *pointer_name;
```

- &a gives the address of a.
- p stores that address.

```
int a = 10;
int *p = &a;
```

- *p accesses the value stored at that address, which is 10.

Pointers

& is the address-of operator used to get a variable's memory address

* is the dereference operator used to access the value stored at a pointer's address.

Variable Name

a

Stores Value

10

Address

&a = 0x1000

Pointer

p

Stores Address

&a

Accesses the value stored at that address

*p

Defining and initializing pointers

- The pointer type indicates the type of object it is intended to point to.
- Initialization:

```
int *p;

float *q;

char *r;
```

- A pointer can also be initialized to NULL when it does not yet point to a valid object:

```
int *p = NULL;
```

- Do not dereference a null or uninitialized pointer.

Pointer operators

1. Address-of operator (&): Returns the address of an object.
2. Dereference operator (*): Accesses the object pointed to by a valid pointer.

```
int a = 10;
printf("%p", (void *)&a);
```

0x7ffd46b6404c

```
int a = 10;
int *p = &a;
printf("%d", *p);
```

10

➢The address is commonly displayed in hexadecimal notation. The actual address varies between executions and systems.

Recap !

- & asks: Where is the variable?
- * asks: What value is stored there?

Expression	Meaning
a	Value of a
&a	Address of a
p	Address stored in pointer p
*p	Value of the object pointed to by p

Modifying a variable using a pointer

- A pointer allows us to access and modify a variable indirectly through its address.
- p stores the address of a; *p = 50; writes 50 to the object at that address.

```c
#include <stdio.h>
int main()
{
    int a = 10;
    int *p = &a;
    printf("Before: %d\n", a);
    *p = 50;
    printf("After: %d\n", a);
    return 0;
}
```

```
Before: 10
After: 50
```

Pointer expressions

- A pointer expression is an expression involving a pointer variable and operations such as assignment, dereferencing, increment, or comparison.

```
p = &a;          // Store the address of a
printf("%d", *p);  // Display the value of a
*p = 20;         // Change a to 20
```

- The expression *p = 20; modifies the value of the pointed-to object, not the address stored in p.

Pointer arithmetic

- Pointer arithmetic allows a pointer to move between elements of an array.

```
int a[3] = {10, 20, 30};
int *p = a;
```

- Initially, p points to a[0]; p + 1 points to a[1]; p + 2 points to a[2]; and p++ advances the pointer to the next integer element.
- The pointer does not necessarily increase by one byte. It advances by the size of the pointed-to type.
- For example, if an int occupies 4 bytes, moving from one integer element to the next advances the address by 4 bytes on that system.
- If p = &a[0], then *p is 10, *(p + 1) is 20, and *(p + 2) is 30.

Pointer Arithmetic: Example

- p = a; makes p point to the first element; *p prints 10; The first p++ moves to the second element; The next p++ moves to the third element.
- Pointer arithmetic must remain within the same array or one position past its end. The one-past-the-end pointer must not be dereferenced.
- Array name is not a modifiable pointer variable. So, we cannot write a++

```c
#include <stdio.h>
int main()
{
    int a[3] = {10, 20, 30};
    int *p = a;
    printf("%d\n", *p);
    p++;
    printf("%d\n", *p);
    p++;
    printf("%d\n", *p);
    return 0;
}
```

```
10
20
30
```

Pointers and one-dimensional arrays

Dr. Aguru Aswani Devi

Pointers and one-dimensional arrays

- The array name a is converted to a pointer to its first element.

- Therefore, p = a; is equivalent to p = &a[0];.

```c
#include <stdio.h>
int main()
{
    int a[4] = {10, 20, 30};
    int *p = a;
    printf("%d\n", a[0]);
    printf("%d\n", *a);
    printf("%d\n", a[1]);
    printf("%d\n", *(a + 1));
    printf("%d\n", p[1]);
    printf("%d\n", *(p + 1));
    return 0;
}
```

```
10
10
20
20
20
20
```

Accessing array elements using pointers

```c
#include <stdio.h>
int main()
{
    int a[5] = {10, 20, 30, 40, 50};
    int *p = a;
    int i;
    for (i = 0; i < 5; i++)
    {
        printf("%d \n", *(p + i));
    }
    return 0;
}
```

```
10
20
30
40
50
```

a[i] = *(a+i)

Dry Run

Expression	Equivalent expression	Value
*(p + 0)	a[0]	10
*(p + 1)	a[1]	20
*(p + 2)	a[2]	30
*(p + 3)	a[3]	40
*(p + 4)	a[4]	50

Modifying array elements using pointers

```c
#include <stdio.h>
int main()
{
    int a[3] = {10, 20, 30};
    int *p = a;
    *(p + 1) = 100;
    printf("%d %d %d", a[0], a[1], a[2]);
    return 0;
}
```

10 100 30

- *(p + 1) refers to the second element of the array.
- Assigning 100 to it changes a[1].

String operations

Dr. Aguru Aswani Devi

String operations

- A string is a sequence of characters terminated by the null character, '\0', which marks the end of the string.

```
char name[] = "ASWANI";
```

- The compiler stores the characters as follows:

A	S	W	A	N	I	\0
Index 0	Index 1	Index 2	Index 3	Index 4	Index 5	Index 6

- The string "ASWANI" requires 6+1 = 7 character positions, including the terminating '\0'.

Declaring and initializing strings

1. `char name[10];` ⇒ This declares space for ten characters. The array is not automatically initialized when declared as an ordinary local variable.
2. `char name[] = "Hello";` ⇒ The compiler allocates six character positions: five letters and the terminating null character.
3. `char name[6] = {'H', 'e', 'l', 'l', 'o', '\0'};` ⇒ This explicitly initializes the character array as a string.

Reading and displaying strings

- %s is used to read or display a string.
- name is passed without & because the array name is converted to a pointer to its first element in this function call.
- %29s limits input to at most 29 characters, leaving space for '\0' in the 30-character array.
- scanf("%s", ...) stops at whitespace, so it cannot read a full name containing spaces.

```c
#include <stdio.h>
int main()
{
    char name[30];
    printf("Enter your name: ");
    scanf("%29s", name);
    printf("Name = %s", name);
    return 0;
}
```

```
Enter your name: aswani
Name = aswani
```

Read a line containing spaces

- fgets() may retain the newline character if there is room in the array.

```
char name[50];

printf("Enter your full name: ");
fgets(name, sizeof(name), stdin);
```

Standard string functions

- The header file <string.h> provides common string-processing functions.

Function	Purpose	Example
strlen()	Finds string length, excluding '\0'	strlen(s)
strcpy()	Copies one string to another	strcpy(dest, src)
strcat()	Appends one string to another	strcat(dest, src)
strcmp()	Compares two strings	strcmp(s1, s2)

strlen() — length of a string

- In C, %zu prints a size_t value, which is the unsigned integer type returned by sizeof and used for sizes and counts.
- strlen() counts the characters before '\0'. It does not count the terminating null character.

```c
#include <stdio.h>
#include <string.h>
int main()
{
    char s[] = "Hello";
    printf("Length = %zu", strlen(s));
    return 0;
}
```

Length = 5

strcpy() — copying a string

- The destination must have enough space for the source string, including '\0'.

```c
#include <stdio.h>
#include <string.h>
int main()
{
    char source[] = "Hello";
    char destination[20];
    strcpy(destination, source);
    printf("%s", destination);
    return 0;
}
```

Hello

strcat() — joining strings

- strcat() appends the second string to the first. The destination array must have enough remaining capacity.

```c
#include <stdio.h>
#include <string.h>
int main()
{
    char first[30] = "Good";
    char second[] = " Morning";
    strcat(first, second);
    printf("%s", first);
    return 0;
}
```

Good Morning

strcmp() — comparing strings

- If the return value is:
  - 0: the strings are equal
  - -ve: the first string compares less than the second
  - +ve: the first string compares greater than the second.
- Do not use s1 == s2 to compare string contents. That expression compares pointer values after array-to-pointer conversion, not the character sequences.

strcmp() — comparing strings

```c
#include <stdio.h>
#include <string.h>
int main()
{
    char s1[] = "apple";
    char s2[] = "apple";
    printf("%d", (strcmp(s1, s2)));
    return 0;
}
```

0

```c
#include <stdio.h>
#include <string.h>
int main()
{
    char s1[] = "apple";
    char s2[] = "apples";
    printf("%d", (strcmp(s1, s2)));
    return 0;
}
```

-115

```c
#include <stdio.h>
#include <string.h>
int main()
{
    char s1[] = "apples";
    char s2[] = "apple";
    printf("%d", (strcmp(s1, s2)));
    return 0;
}
```

115